

Data Structures and Algorithms

Data structures and algorithms are fundamental concepts in computer science used to organize data and solve computational problems efficiently. Efficient algorithms improve application performance, scalability, and resource usage.

Arrays

Arrays store elements in contiguous memory locations and allow constant-time access using indexes. Arrays are commonly used for storing ordered collections of data.

Linked Lists

Linked lists consist of nodes connected through pointers. Unlike arrays, linked lists allow efficient insertions and deletions without shifting elements in memory.

Stacks

Stacks follow the Last In First Out principle. Elements are added and removed from the top of the stack. Common applications include function call management, undo systems, and expression evaluation.

Queues

Queues follow the First In First Out principle. Elements are added at the rear and removed from the front. Queues are widely used in scheduling systems, task processing, and breadth-first search.

Hash Tables

Hash tables store key-value pairs and provide average constant-time lookup, insertion, and deletion. Hash functions map keys to storage locations efficiently.

Trees

Trees are hierarchical data structures composed of parent and child nodes. Binary trees, binary search trees, AVL trees, and heaps are commonly used variants.

Binary Search Trees

Binary search trees maintain ordered data such that left child nodes are smaller and right child nodes are larger than the parent node. This enables efficient searching, insertion, and deletion.

Heaps and Priority Queues

Heaps are specialized tree structures used to implement priority queues. Min-heaps and max-heaps efficiently retrieve the smallest or largest element.

Graphs

Graphs represent relationships between entities using vertices and edges. Graphs are used in social networks, routing systems, recommendation engines, and dependency analysis.

Graph Traversal

Depth-first search explores nodes by going as deep as possible before backtracking.

Breadth-first search explores nodes level by level using a queue.

Sorting Algorithms

Sorting algorithms arrange data into a specific order. Common sorting algorithms include merge sort, quicksort, bubble sort, insertion sort, and heap sort.

Merge Sort

Merge sort uses divide-and-conquer to recursively split arrays and merge sorted halves. It guarantees $O(n \log n)$ time complexity.

Quick Sort

Quick sort selects a pivot and partitions elements around it. It is highly efficient on average but may degrade to $O(n^2)$ in worst-case scenarios.

Binary Search

Binary search efficiently finds elements in sorted arrays by repeatedly dividing the search space in half. Its time complexity is $O(\log n)$.

Dynamic Programming

Dynamic programming solves complex problems by breaking them into overlapping subproblems and storing intermediate results to avoid redundant computation.

Greedy Algorithms

Greedy algorithms make locally optimal decisions at each step in hopes of finding a globally optimal solution. Examples include interval scheduling and Huffman coding.

Time and Space Complexity

Algorithm efficiency is analyzed using Big O notation. Time complexity measures execution growth relative to input size, while space complexity measures memory usage.

Conclusion

Data structures and algorithms are essential for building efficient software systems. Understanding computational complexity and selecting appropriate data structures are critical skills for scalable software engineering.